

# CSE 1113

# Computer Programming

Sumaya Kazary

Assistant Professor

Department of Computer Science and Engineering  
Dhaka University of Engineering & Technology, Gazipur

## Acknowledgement

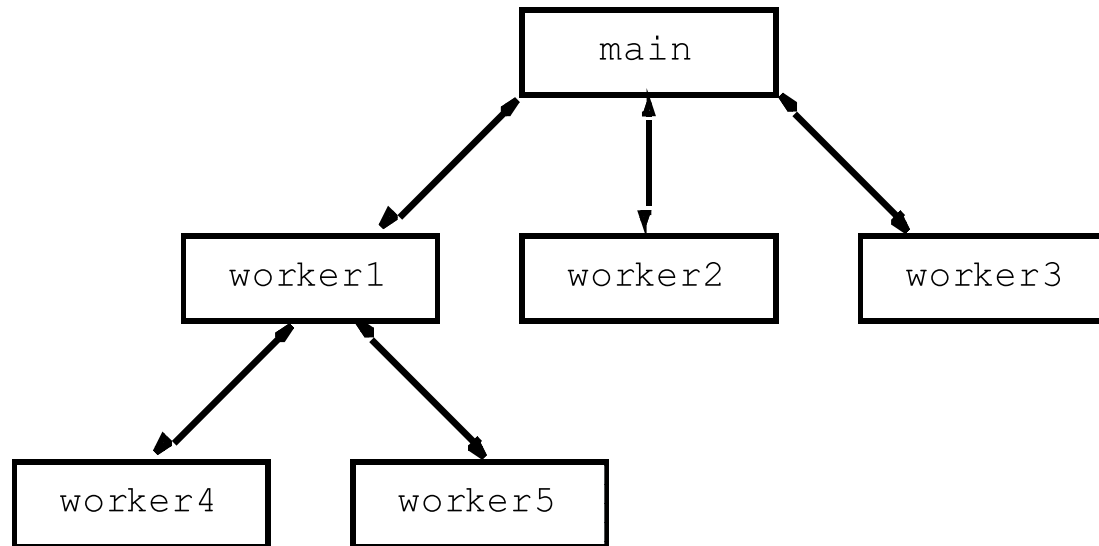
Thanks to the authors of all the books and online tutorials used in this slide.

# Functions in C

# Introduction

- Programs are typically very large, much larger than what we have seen so far!
- **Divide and conquer** technique
  - Construct a program from smaller pieces or components
  - Each piece more manageable than the original program
- **Modular programming design**
  - Large and complex task can be divided into smaller and simple task which is more easily solved(implemented).
  - Also called
    - structured design
    - Top-down design
    - Divide-and-Conquer

# Program Modules in C



**Fig. Hierarchical boss function/worker function relationship.**

## Function call analogy:

- Boss asks worker to complete task
  - Worker gets information, does task, returns result
  - Information hiding: boss does not know details

# Introduction

- A function is a series of statements that have been **grouped together and given a name.**
- Each function is essentially a small program, with its own declarations and statements.
- **Advantages of functions:**
  - A program can be divided into small pieces that are easier to understand and modify.
  - We can avoid duplicating code that's used more than once.
  - A function that was originally part of one program can be reused in other programs.

# What is a Function?

- A small program(subroutine) that performs a particular task
  - **Input** : parameter / argument
  - **Perform what ?** : function body
  - **Output** : return value

# Types of Functions

→ There are two types of functions:

- Library functions
- Programmer/User Defined functions

# Library Functions

- C standard library provides a rich collection of functions for performing ~~~
  - ✓ I/O operations,
  - ✓ Mathematical Calculations,
  - ✓ String manipulation operations etc.
- For example, **sqrt(x)** is a function to calculate the square root of a double number provided by the C standard library and included in the **<math.h>** header file.



# Library Functions

## Example:

:

```
double a=9.9, b;
```

```
b = sqrt(a);
```

```
printf("The square root of %f is %f", a, b);
```

:

- Other functions such as **exp(x)** (exponential function  $e^x$ ) and **pow(x,y)** ( $x^y$ ) ... can be used as they are needed.
- Note that each program in C has a function called main which is used as the root function of calling other library functions.

# Programmer/user Defined Functions

- In C, the programmers can write their own functions and use them in their programs.

- For Example:

The program on the next slide calls the user defined function called *square* to calculate the square of the numbers from 1 to 10.

# User Defined Functions

```
#include<stdio.h>
```

```
int square( int);
```

```
void main() {
```

```
    int x;
```

```
    printf("the squares of numbers from 1 to 10 are:\n");
```

```
    for(x=1 ; x<= 10; x++)
```

```
    { y = square(x);
```

```
        printf("the square of %d = %d\n",x, y);
```

```
    }
```

```
}
```

```
/*function definition */
```

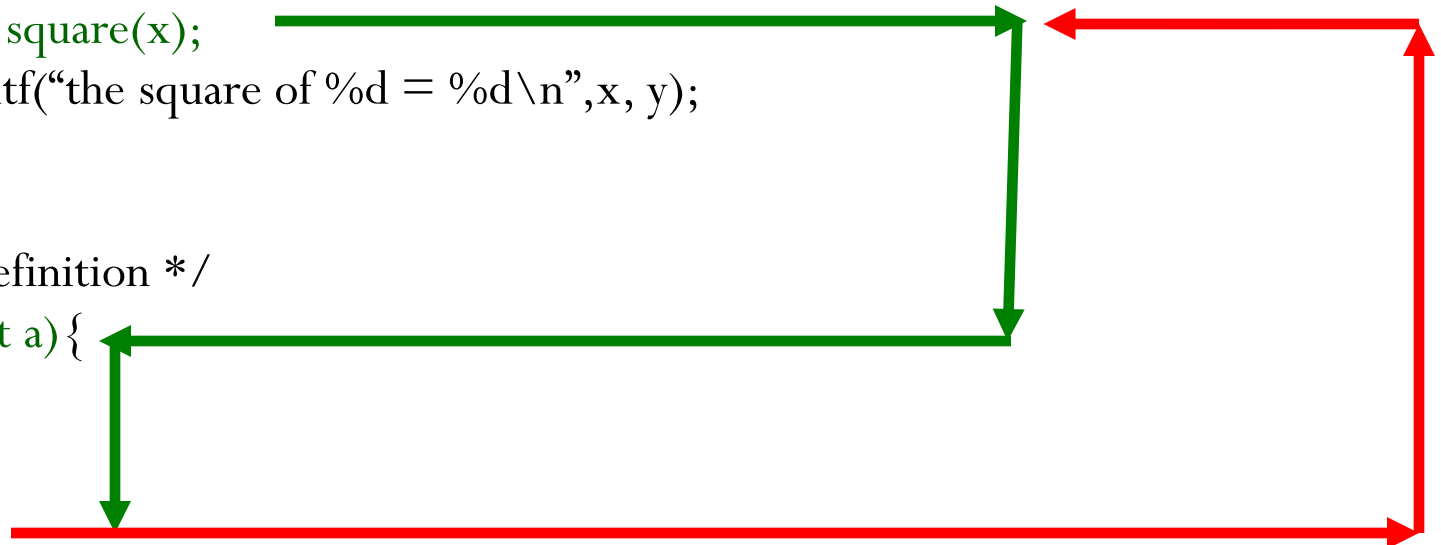
```
int square (int a){
```

```
    int b;
```

```
    b = a * a;
```

```
    return b;
```

```
}
```



# Definitions

- *Local variables*
  - Known only in the function in which they are defined
  - All variables declared in function definitions are local variables
- *Parameter*:— a declaration of identifier(s) within the ' ( ) ' of a function declaration
  - Used within the body of the function as a *variable* of that function
  - Initialized *by the caller* to the value of the corresponding *argument*.
- *Argument*:— an expression passed when a function is *called*; becomes the initial value of the corresponding parameter

# User Defined Functions

```
#include<stdio.h>
```

```
int square( int);
```

```
void main() {
```

```
    int x;
```

```
    printf("the squares of numbers from 1 to 10 are:\n");
```

```
    for(x=1 ; x<= 10; x++)
```

```
        { y = square(x);
```

```
          printf("the square of %d = %d\n",x, y);
```

```
        }
```

```
    }
```

```
/*function definition */
```

```
int square (int a){
```

```
    int b;
```

```
    b = a * a;
```

```
    return b;}
```

Argument/ Actual Parameter

Parameter/ Formal  
Parameter

# Function Definitions (How to define a Function)

- Format/Syntax for function definition

```
return-value-type function-name( parameter-list )  
{  
    declarations and statements;  
}
```

- *Parameter list*

- Comma separated list of arguments
  - Data type needed for each argument
- If no arguments, use **void** or leave blank

- *Return-value-type*

- Data type of result returned
- Use **void** if nothing returned

# Function Prototype

- Function prototypes are **always** declared at the beginning of the program indicating
  - the **name of the function**,
  - the **data type of its arguments** (which is passed to the function),
  - the **data type of the returned value** from the function.

Ex: `int square( int)`

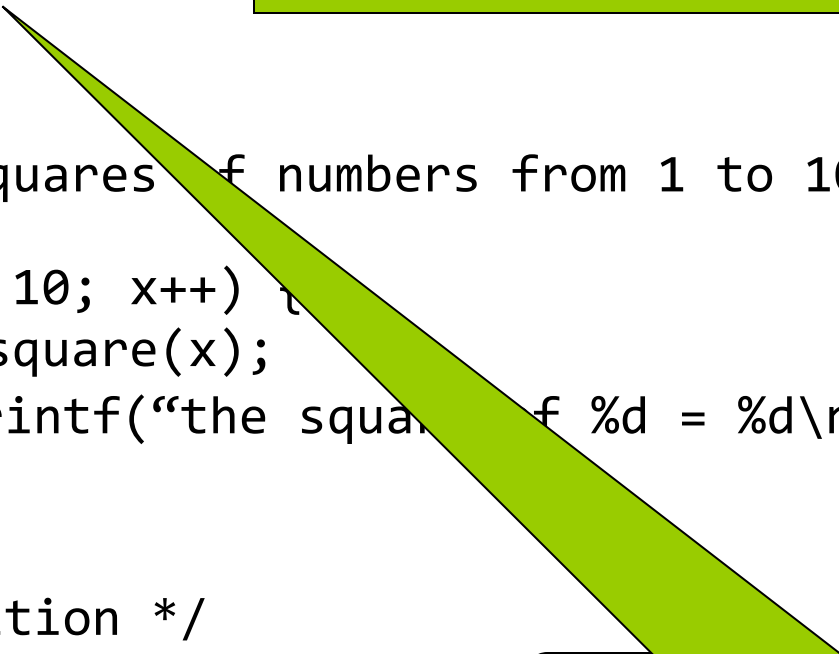
The name of the function

The data type of the value returned from the function

The data type of the argument passed to the function

# Function Prototype

```
#include<stdio.h>
int square(int);
void main() {
    int x;
    printf("the squares of numbers from 1 to 10
        are:\n");
    for(x=1 ; x<= 10; x++) {
        y = square(x);
        printf("the square of %d = %d\n",x, y);
    }
}
/*function definition */
int square (int a){
    int b;
    b = a * a;
    return b;
}
```



/\* function prototype \*/

Don't forget to put this ;



# Purposes of Function Prototype

- So compiler knows how to compile calls to that function, i.e.,
  - Number and types of arguments
  - Type of result
- As part of hiding details of *how* it works and exposing *what* it does.
- A function serves as a “black box.”

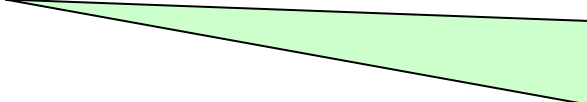
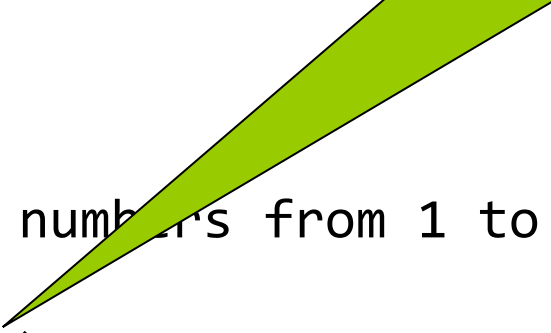
# Calling/invoking a function (in the program)

- Parentheses an operator used to call function
  - Pass argument
  - Function gets its own copy of arguments
  - After finished, passes back result
- When a function prototype is defined, the function has to follow that prototype:
  - i.e., the return type cannot change (compilation error),
  - the parameter list has to be in the order it was defined.
  - If no parameters, use **void** between **()**

# Calling a function

```
#include<stdio.h>
int square( int);
void main() {
    int x;
    printf("the squares of numbers from 1 to 10 are:\n");
    for(x=1 ; x<= 10; x++)
        {   y = square(x);
            printf("the square of %d = %d\n",x, y);
        }
}

/*function definition */
int square (int a){
    int b;
    b = a * a;
    return  b;
}
```



**function call**

**Definition of square.**  
**a** is a copy of the argument passed.  
Returns a \* a, or **a** squared.

**average.c:** Computes pair wise averages of three numbers

```
#include <stdio.h>

double avg(double a, double b){
    return (a + b) / 2.0;}

void main(){
    double x, y, z;

    printf("Enter three numbers: ");
    scanf("%lf %lf %lf", &x, &y, &z);

    printf("Average of %lf and %lf:%lf\n",x,y,avg(x,y));
    printf("Average of %lf and %lf:%lf\n",y,z,avg(y,z));
    printf("Average of %lf and %lf:%lf\n",x,z,avg(x,z));
}
```

# Program: Computing Averages

- The `average.c` program reads three numbers and uses the `average` function to compute their averages, one pair at a time:

```
Enter three numbers: 3.5 9.6 10.2
```

```
Average of 3.5 and 9.6: 6.55
```

```
Average of 9.6 and 10.2: 9.9
```

```
Average of 3.5 and 10.2: 6.85
```

# Finding the maximum of three integers

```
#include <stdio.h>
```

```
int maximum( int x, int y, int z );          /* function prototype */
```

```
void main(){
```

```
    int number1;          /* first integer */
```

```
    int number2;          /* second integer */
```

```
    int number3;          /* third integer */
```

```
    printf( "Enter three integers: " );
```

```
    scanf( "%d%d%d", &number1, &number2, &number3 );
```

```
/*number1, number2 and number3 are arguments to the maximum function  
call*/
```

```
    printf( "Maximum is: %d\n", maximum( number1, number2, number3 ) );
```

```

/* Function maximum definition */
int maximum( int x, int y, int z ) {    /* x, y and z are parameters */
    int max = x;    /* assume x is largest */
    if ( y > max ) {
        max = y;
    } /* end if */
    if ( z > max ) {
        max = z;
    } /* end if */
    return max;    /* max is largest value */
} /* end function maximum */

```

```

Enter three integers: 22 85 17
Maximum is: 85
Enter three integers: 85 22 17
Maximum is: 85
Enter three integers: 22 17 85
Maximum is: 85

```

# Header Files

- Header files
  - Contain function prototypes for library functions
  - `<stdlib.h>` , `<math.h>` , etc
  - Load with `#include <filename>`  
`#include <math.h>`
- Custom header files
  - Create file with functions
  - Save as `filename.h`
  - Load in other files with `#include "filename.h"`
  - Reuse functions



# Questions?